

BACHELORARBEIT

Implementation of combinatorial n-fold Algorithms

Lennart Klose

Abgabe Monat + Jahr

Betreut durch:
Prof. Dr. Klaus Jansen
Lis Piroton

Institut für Informatik
Arbeitsgruppe Algorithmen und Komplexität
CHRISTIAN-ALBRECHTS-UNIVERSITÄT ZU KIEL

Eidesstattliche Erklärung

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Kiel, Datum

Abstract

Contents

1	Introduction	1
2	Background	3
3	Methodology	5
3.1	Implementation of Algorithms	5
3.1.1	Definitions	5
3.1.2	Lis	6
3.1.2.1	Description	6
3.1.2.2	Implementation	6
3.1.3	Lars1	9
3.1.3.1	Description	9
3.1.3.2	Implementation	9
3.1.4	Gupta	11
3.1.4.1	Description	11
3.1.4.2	Implementation	12
3.1.5	Lars2	13
3.1.5.1	Description	13
3.1.5.2	Implementation	14
4	Conclusion	17
	Bibliography	19

1

Introduction

2

Background

3

Methodology

3.1 Implementation of Algorithms

For all of the implemented algorithms, an ILPInstance class was created that stores information about an ILP problem. Namely, it stores the input parameters *params* that were used to generate the problem, where

$$params = [max_A, max_B, delta, blockamount, blockwidth, r]$$

as well as the values for A, b, n, r, h, t and c . This class provides simple getter methods which allow each of the subsequent algorithms to retrieve the input information about a singular instance in constant time $\mathcal{O}(1)$. Those instances were generated using a python script (citation needed) and then parsed using an InstanceParser class, which in turn generates several instances of ILPInstance. It is assumed that each implemented algorithm first parses a generated instance from a file and trivially extracts the relevant values in constant time for further processing. Furthermore, unless specified otherwise, vectors will be represented by an (integer) array in the actual Java code. Additionally, reused variables are declared outside of loops whenever applicable. Often times in the algorithms, we will describe sets such as $b_1, \dots, b_n$ with $n \in N$, but the code will be indexed starting at 0, as is traditionally the case in most programming languages.

3.1.1 Definitions

Definitions (put this in another chapter):

$\mathcal{A} \in \mathbb{N}^{m \times tn}$ = the whole matrix

A = upper section

$A_1, \dots, A_n \in \mathbb{N}^{r \times t}$ = Matrices/Blocks

n = number of blocks t = blockwidth nt = columns

m = rows

B = lower section

$(\spadesuit) = \mathcal{A}x = b$

Δ = largest value in $\mathcal{A}$

$b \in \mathbb{N}^{r+n}$ = rhs

$b^\uparrow \in \mathbb{N}^r, b^\downarrow \in \mathbb{N}^n$ = upper and lower RHS

3.1.2 Lis

3.1.2.1 Description

The algorithm demonstrated in [1] solves the n -fold ILP by dividing the large and complex problem into several smaller sub-problems. Matrix–vector multiplication can be interpreted as forming a linear combination of vectors, where the columns of A are the vectors and the corresponding entries of x determine the coefficient by which each vector is scaled, or in other terms, how often we add that specific vector to the coordinate grid. The paper shows that there exists a solution to ($\spadesuit$) which can be constructed by doubling a solution close to $b^\uparrow/2$, where “close” means that the absolute difference between each coordinate of the vector and the corresponding coordinate of $b^\uparrow/2$ is at most D , and subsequently adding the solution of a small sub-problem. This idea can be used repeatedly and iteratively on itself, where we keep finding solutions by halving the distance and adding an offset, until the zero vector $\mathbf{0}$ is reached.

The algorithm operates by solving two subproblems. The first subproblem is called the *parity constrained* problem, which contains only even entries in its lower right hand side $\hat{b}^\downarrow[1]$. The second subproblem is called the *small* problem, with lower right hand side $\tilde{b}^\downarrow$. This problem is defined by $\|\tilde{b}^\downarrow\|_\infty \leq K$. The algorithm goes through $\mathcal{I}$ iterations, where for each iteration $i \in [\mathcal{I}]$ it holds true that $b_k^{\downarrow(i)} = \tilde{b}_k^{\downarrow(i)} + \hat{b}_k^{\downarrow(i)}$ for every $k \in [n]$.

To determine feasibility, we need to build two set of sets $\tilde{N}$ and $\hat{N}$ which get combined into N . Specifically, each set has $\mathcal{I}$ entries of different sets which are defined as follows:

$$\tilde{N}^{(i)} = \left\{ \tilde{b}^{\uparrow(i)} \in \mathbb{Z}_{\geq 0}^r \mid A\tilde{x}^{(i)} = (\tilde{b}^{\uparrow(i)}, \tilde{b}^{\downarrow(i)})^T, \tilde{x}^{(i)} \in \mathbb{Z}_{\geq 0}^m, \tilde{b}^{\uparrow(i)} \leq D \right\},$$

and

$$\hat{N}^{(i)} := \left\{ \hat{b}^{\uparrow(i)} \in \mathbb{Z}_{\geq 0}^r \mid \hat{b}^{\uparrow(i)} = 2b^{\uparrow(i-1)}, b^{\uparrow(i-1)} \in N^{(i-1)} \right\}.$$

$\tilde{N}^{(i)}$ can be constructed with a dynamic program with the lower right hand side $\tilde{b}^{\downarrow(i)}$ as the input [1]. Finally, $N^{(i)}$ is defined as

$$N^{(i)} := \left\{ b^{\uparrow(i)} \in \mathbb{Z}_{\geq 0}^r \mid b^{\uparrow(i)} = \hat{b}^{\uparrow(i)} + \tilde{b}^{\uparrow(i)}, \hat{b}^{\uparrow(i)} \in \hat{N}^{(i)}, \tilde{b}^{\uparrow(i)} \in \tilde{N}^{(i)}, \|b^\uparrow/2^{\mathcal{I}-i} - b^{\uparrow(i)}\|_\infty \leq D \right\}.$$

For the construction to work, we set $N^{(1)} := \tilde{N}^{(1)}$ as the set of feasible points after this iteration [1]. To determine feasibility, in iteration $\mathcal{I}$, we check if $b^\uparrow$ is contained in the set $N^{(\mathcal{I})}$

3.1.2.2 Implementation

The implementation starts by trivially obtaining relevant values that are needed for the process, such as $n, \Delta, K, D, b_{max}^\downarrow$ and r . To determine the total number of

iterations $\mathcal{I}$, which is critical for the algorithm, we use Lemma 11 [1] which gives us a result in constant time, where it holds that

$$\mathcal{I} = \begin{cases} \log((b_{\max}^{\downarrow} + K)/(2K + 1)) + 2, & \text{if } \log((b_{\max}^{\downarrow} + K)/(2K + 1)) \text{ is an integer,} \\ \lceil \log((b_{\max}^{\downarrow} + K)/(2K + 1)) \rceil + 1, & \text{otherwise.} \end{cases}$$

With those values obtained, we ultimately want to construct $N^{(i)}$, for which we first need to determine the sets $\tilde{N}^{(i)}$ and $\hat{N}^{(i)}$. Due to the nature of their definitions, we need to construct $\tilde{N}^{(i)}$ first. For this purpose, we can use Algorithm 1 (citation needed) as described in the paper by the original authors. This implementation, however, requires $\tilde{b}^{\downarrow(i)}$ as the input. For this purpose, the implementation first determines $b^{\downarrow(i)}$ for all $i \in [\mathcal{I}]$. This is done by working backwards through the iterations. We know that $b^{\downarrow(\mathcal{I})} = b^{\downarrow}$ as in the final iteration we have reached our b . With that information in mind, we determine the value for the lower RHS by checking each component of the following iterations individually. For each $i \in [\mathcal{I} - 1]$ and $k \in [n]$ we then check $b_k^{\downarrow(i+1)}$. If $b_k^{\downarrow(i+1)} \leq K$ we set $b_k^{\downarrow(i)}$ to 0. Otherwise, we can derive it as follows:

$$b_k^{\downarrow(i)} = \frac{b_k^{\downarrow(i+1)} - (K + z_k^{(i+1)})}{2}$$

where

$$z_k^{(i+1)} = \begin{cases} 0, & \text{if } b_k^{\downarrow(i+1)} \bmod 2 = K \bmod 2, \\ 1, & \text{otherwise.} \end{cases}$$

After this pre-processing step, we still need to extract the *small* $\tilde{b}^{\downarrow(i)}$ component from every $b^{\downarrow(i)}$. This is done by once again going over every $b_k^{\downarrow(i+1)}$ with $i \in [\mathcal{I}], k \in [n]$ and checking if the value is $\leq K$. If yes, we can immediately set $\tilde{b}_k^{\downarrow(i+1)} = b_k^{\downarrow(i+1)}$. If not, we know that $b_k^{\downarrow(i+1)} = K$ if $(b_k^{\downarrow(i+1)} - K) \bmod 2 = 0$ and $K - 1$ otherwise. With this step finalized, we can implement Algorithm 1 (citation needed) to generate $\tilde{N}^{(i)}$ for a given $\tilde{b}^{\downarrow(i)}$, which in the code will be referred to as *buildSmallRHS*. The algorithm defines two different tables, a base table $BT(v, k)$ where

$$BT(v, k) \leftarrow \begin{cases} \text{true,} & \text{if } A_k \tilde{x}_k = v \text{ with } \|\tilde{x}_k\|_1 = \tilde{b}_k^{\downarrow(i)}, \tilde{x}_k \in \mathbb{Z}_{\geq 0}^{t_i} \text{ is feasible,} \\ \text{false,} & \text{otherwise.} \end{cases}$$

and a dynamic table $DT(v, k)$ with

$$DT(v, k) \leftarrow \bigvee_{\substack{\nu' + \nu'' = v \\ \nu', \nu'' \text{ integral vectors}}} DT(\nu', k - 1) \wedge BT(\nu'', k)$$

where $DT(v, 1) = BT(v, 1)$. Additionally, the components of each vector in the tables are bounded by $K\Delta$ for the base table, and by D for the dynamic table. As we are exclusively working with vectors that contain non-negative entries, and we know the largest value each component of the vectors can achieve in each table, we can interpret a vector as a number read in base $K\Delta + 1$ and base $D + 1$ and store it as an integer. For example, using base 4, the vector $\{1, 3, 2\}$ can be encoded as the integer $1 \cdot 4^2 + 3 \cdot 4 + 2 = 30$. By encoding the vectors we can use the Java BitSet

class (citation needed) for a memory-efficient storage method. A bitset is a data structure that stores Boolean values as individual bits, with each bit representing either a true or false value. As our tables represent true or false values, this data type is an excellent choice for our purpose. However, the encoding comes with some runtime overhead, needing a linear runtime in the length of the vector. This tradeoff was deemed worth it, as the memory limitations proved to be a major bottleneck for actual testing. As in each iteration $k \in [n]$ we only need to know the values from the tables for k and $k - 1$, we can smartly keep reusing the same BitSets to not store unnecessary information. Notably, we have the BitSets BT , DT and DT_{Prev} . The latter is needed to access information of the previous iteration $k - 1$.

Initially, the algorithm determines the feasibility of $A_k \tilde{x}_k = v$ with $\|\tilde{x}_k\|_1 = \tilde{b}_k^{\downarrow(i)}$, $\tilde{x}_k \in \mathbb{Z}_{\geq 0}^t$ for all $v \in \{0, \dots, K\Delta\}$ and $k = 1$. To fulfill the secondary condition of $|\tilde{x}_k|_1 = \tilde{b}_k^{\downarrow(i)}$, a row of $\mathbf{1}^t$ was appended to each A_k . Since $\tilde{x}_k$ contains only non-negative entries, this row ensures that their sum, and hence $\|\tilde{x}_k\|_1$, is constrained to $\tilde{b}_k^{\downarrow(i)}$. The chosen way to determine feasibility for this small problem was the Lars1 (TODO: CHANGE NAME) algorithm, which is also implemented as part of this paper. If a vector v turned out to be feasible, we set the bit in BT and DT_{Prev} . Now we can work through the next iterations intelligently. Recall the definition of $DT(\nu, k)$. Instead of considering every possible decomposition $v = v' + v''$, we only consider combinations for which v' is already set in DT_{Prev} and v'' is set in BT . The set bits in BT and DT_{Prev} can be identified in linear time. Their corresponding indices are then decoded back into vectors, which are added component-wise. If every component of the resulting vector satisfies the bound $\leq D$, the corresponding bit is set in DT . Once all valid combinations have been processed, BT is cleared for the next iteration, and DT_{Prev} is updated to DT . Finally, DT is cleared to prepare it for the next iteration. Once every iteration k finished, we return an ArrayList of each bit that was set in DT_{Prev} decoded back into a vector.

Now we are finally ready to construct $N^{(i)}$. For this we store three ArrayList variables, N_{curr} , N_{prev} and N_{small} . We determine N^1 by using *buildSmallRHS* and setting value of N_{small} and N_{prev} to the result of the function call. For every iteration $i \geq 2$ we then determine N_{small} by another *buildSmallRHS* call, with an updated iteration parameter, and then we build $N^{(i)}$ by combining the vectors v' and v'' from N_{prev} and N_{small} respectively. We check every possible combination of vectors $v = 2v' + v''$ and check every component of the vector to see if $\|2^{I-i}v - b^{\uparrow(i)}\|_{\infty} \leq D$. We are scaling the vectors rather than dividing them to avoid floating point arithmetic and the possible inaccuracies that result of it. If the vector v satisfied the condition and is not already present in N_{curr} , we add the entry to N_{curr} . This takes a linear lookup of the ArrayList each time, however, alternative set solutions with indexing have found to require too much overhead to be worth the cost. After we checked all combinations, we update N_{prev} to N_{curr} and clear N_{curr} . Once all iterations are finished, we can determine feasibility by checking if $b^{\uparrow}$ is contained in $N^{(I)}$. If yes, the n -fold ILP is feasible.

3.1.3 Lars1

3.1.3.1 Description

The algorithm once again benefits from viewing vectors as coordinates in a grid $\mathbb{N}^r$. Given a solution x^* , we know that the vector $\mathcal{A}_i$ gets added x_i^* times, with the resulting sum equal to b . If we add these vectors one at a time in some order, the resulting sequence of partial sums forms a walk from the origin to b . The Steinitz Lemma provides the intuition that, if each vector is bounded with respect to some norm, the vectors can be reordered such that the resulting walk remains close to the path from 0 to b (citation needed). The algorithm only requires the weaker property that the walk visits some vector b' close to $b/2$. This distance is closely related to the hereditary discrepancy of the matrix $\mathcal{A}$ ($\text{herDisc}(\mathcal{A})$) [2]. The discrepancy of a matrix measures how well its columns can be partitioned into two groups, based on the maximum imbalance across its rows. The hereditary discrepancy extends this concept by considering all submatrices obtained by restricting the columns and taking the maximum discrepancy among them. Then, by guessing the vector b' and solving the problem with $\mathcal{A}x = b'$ and $\mathcal{A}x = b - b'$ independently, the results can be added together to form a solution for $\mathcal{A}x = b$. Using this approach dynamically and merging the solutions at each step, we can use convolution to efficiently find a feasible solution.

"Let $H \geq \text{herdisc}(\mathcal{A})$ be a given upper bound on the hereditary discrepancy on $\mathcal{A}$." [2] The algorithm goes through $\mathcal{I} = \lceil \log_{6/5}(K) \rceil$ iterations, where at each iteration $i \in 0 \cup [\mathcal{I}]$ and every b' with $\|b' - 2^{i-\mathcal{I}} \cdot b\|_\infty \leq 4H$, it decides

$$\text{Does there exist } x \in \mathbb{Z}_{\geq 0}^{nt} \text{ such that } \mathcal{A}x = b' \text{ and } \|x\|_1 \leq \left(\frac{6}{5}\right)^i?$$

The original paper formulates this as an optimization problem. Since we focus on feasibility, we omit the objective function. The crucial piece of information that makes the algorithm work is the knowledge that if b' is feasible in a given iteration i with $\mathcal{A}x = b'$, there exists a b'' in iteration $i - 1$ with $\mathcal{A}x = b''$ and $\mathcal{A}x = b' - b''$ being feasible. More precisely this means:

$$b' \text{ feasible at level } i \implies \exists b'' \text{ such that } b'' \text{ and } b' - b'' \text{ are feasible at level } i - 1.$$

3.1.3.2 Implementation

Initially, we the algorithm calculates the the hereditary discrepancy $\text{herdisc}(\mathcal{A})$ directly. The main part of the calculation will be bound by $H \geq \text{herdisc}(\mathcal{A})$, with an upper bound of $\text{herdisc}(\mathcal{A}) \leq 6\sqrt{m} \cdot \Delta$. However, in practice, using the upper bound as our value for H proved to be quite inefficient. Despite the large exponential runtime $\mathcal{O}(ntm3^{nt})$ of using the naive implementation of calculating the hereditary discrepancy, the resulting bounds ended up significantly smaller and ended up pruning many different states of calculation, ultimately deeming this large preprocessing step worth it. We set $\text{herDisc}(\mathcal{A})$ to H . After some basic extraction for the values Δ, m and calculating $K = \sum_{i=r}^m b_i$, we also naively calculate the amount of iterations $\mathcal{I} = \lceil \log_{6/5}(K) \rceil$.

After those preparatory steps, we can now look at the core idea of the algorithm in more detail. As we know that if b' is feasible in iteration i , there exists b'' with b'' and $b' - b''$ both being feasible in iteration $i - 1$. We can reverse the understanding of this by stating that if b'' and $b' - b''$ are feasible in iteration i , b' must be feasible in iteration $i + 1$. As $b' - b'' + b'' = b'$, we can combine both of the feasible vectors to obtain b' . This algorithm reuses the idea of encoding vectors as integers. Let $\max_b$ be the largest entry in b . In this case, we can use $\max\{\Delta, \max_b\} + 1$ as the base B , as this is sufficient to represent all relevant vectors encountered during the calculation. A simple way to see why Δ is required is to consider iteration $i = 0$. Here, $\|x\|_1 \leq (6/5)^0 = 1$, meaning that the possible vectors directly correspond to the columns of $\mathcal{A}$ and the zero vector $\mathbf{0}$ [2]. Since these vectors form the initial states of the calculation, the encoding must be able to represent every entry occurring in the columns of $\mathcal{A}$, hence the need to account for Δ . Additionally, since our inputs contain only non-negative entries, we can discard any vector b' whose entries exceed the corresponding entries of b . Since the entries can only increase as vectors are added, such a vector can never lead to a solution for b . Thus, the entries of all relevant vectors are bounded by the entries of b , which is why we also account for $\max_b$. Therefore, a base of $\max\{\Delta, \max_b\} + 1$ is sufficient.

We can now start the algorithm proper. We initialize the `prev` BitSet, (citation needed) where we set the 0 bit to represent the zero vector $\mathbf{0}$ and then set the bits which represent the encoded columns of $\mathcal{A}$. Then we start the main loop. To save on memory, we work with two BitSets again, `prev` which we already initialized, and `next` which is declared here. We calculate the scaled b array for each iteration i `bound` by scaling it with $2^{i-\mathcal{I}}$. Then, we perform pairwise convolution by considering all pairs of set bits in the `prev` BitSet. For each of them, we want to consider the their sum as this would represent our b' . However, we need to be careful that while adding them, none of the values get larger than the maximum integer allowed in our base representation, as that vector cannot possibly produce a valid solution. We do this by checking if their addition would cause a carry if they were added as number of our chosen base B . If no carry occurs, we sum up the encoded vectors and perform two more combined checks on each component of the `sum` vector. For this, we have to decode the each component of the sum first as we go, so we can make proper comparisons again. As each component of `sum` is decoded to represent b'_k again, we use the `bound` we calculated earlier to see if $\|b'_k - \text{prev}_k\| > 4H$, which corresponds to $|b'_k - 2^{i-\mathcal{I}} \cdot b_k| > 4$. If that condition is true, we can discard the result immediately and skip checking the remaining components. We then perform a second check, which is a further optimization that we can do thanks to nature of our non-negative values. As discussed previously, we cannot ever exceed values in a component that grow larger than the ones in b , as such, we check $b'_k > b_k$. If this check is true, we can equally discard b' at this step. If all checks end up negative, the current b' is a true candidate and the encoded `sum` bit gets set in the `next` BitSet. Once all combinations are checked, we update `next` to `prev`. Ordinarily we would now start the next iteration until all $\mathcal{I}$ iterations are concluded, but we can instead perform the biggest optimization step of the implementation. Here, we can potentially skip several iterations entirely, and as each iteration gets exponentially more computationally expensive, the potential time saved is enormous. Note that in

each iteration, we check the combinations of all set bits. Crucially, we will check the sum for $b'' = 0$ and $b' - b'' = 0$, where the sum is trivially 0. This vector is guaranteed to stay in the BitSet in the next iterations, as the bound condition only becomes less strict. This is also true for all other vectors b' that get added in iteration i , as they are combined with the zero vector $\mathbf{0}$ and are therefore guaranteed to also appear again in the next iteration $i + 1$. That means after each iteration, we can check if the bit for b is set. If yes, it will also be set in the final iteration $\mathcal{I}$. Ultimately this means we can determine feasibility for b before going through all possible iteration steps.

3.1.4 Gupta

3.1.4.1 Description

The algorithm from Gupta [3] uses a rather elegant idea, once more making use of the Steinitz Lemma, to find a solution to the n -fold ILP of the following form

$$\min\{c^T x \mid \mathcal{A}x = b, x \in \mathbb{N}^{nt}\}.$$

This algorithm requires us to introduce several new notations. For a matrix M , $\text{psum}(M, j)$ denotes the partial sum of the first j columns of M . For a matrix M and an entry e , we use $\text{occ}_M(e, j)$ to denote how many times e occurs in M in the first j columns. We define the imbalance of a symbol e at column j in M as

$$\text{imb}_M(e, j) := \text{occ}_M(e, j) - (j/q) \cdot m_e.$$

The imbalance of a symbol e is the difference between how many times e actually appears up until column j , and how many times it would appear if the columns were arranged entirely randomly [3]. We denote the k^{th} columns and the k^{th} row of a matrix M with $M[\cdot, k]$ and $M[k, \cdot]$ respectively. Finally, let σ be a permutation of the columns of M , where M_σ is the permuted matrix.

Suppose that x^* is a solution for our input. In that case, the entries in the lower RHS $b_1^\downarrow, \dots, b_n^\downarrow$ directly specify the amount of vectors chosen from each block $A_i, \dots, A_n$. Let $q = \sum_{i=1}^n b_i^\downarrow$ denote the sum of the entries in $b^\downarrow$. All q columns of $A_i, \dots, A_n$ that are chosen in the solution must, therefore, add up to $b^\uparrow$. Importantly, the column $A_i[\cdot, k]$ appears $x_k^{*(i)}$ many times in the solution. One of the main insights is that the columns chosen in the solution can be arranged in several different ways, where the order affects the intermediate partial sums. Let M be the $1 \times q$ matrix with entries from $[n]$ where $M[j] = i$ if $v_j = A_i[\cdot, k]$ for some $k \in [t]$ [3], where v_j corresponds to the j^{th} column of $\mathcal{A}$ restricted to the first r rows. This matrix identifies from which block each vector is obtained [3].

The key idea is to use the Steinitz Lemma to order the processing of the columns in such a way that the partial sums remain bounded by a constant factor. Gupta shows that there exists a permutation σ of M such that, for every symbol $e \in [n]$ and every column $j \in [q]$, the imbalance is bounded by

$$-n \leq \text{imb}_{M_\sigma}(e, j) \leq 1.$$

Let O denote the matrix obtained by arranging the multiset of columns selected by x^* in the order constructed by Gupta. With this permutation, it holds that for each $j \in [q]$ and $k \in [r]$:

$$-n\Delta(n+2r) \leq \text{psum}(O, j)[k] - \frac{j}{q}b_k^\uparrow \leq n\Delta(1+2r).$$

Thus, every partial sum of O remains within a bounded distance of its proportionally expected value. This bound is the key property exploited by the algorithm: it allows Gupta to construct a weighted DAG G whose states only need to represent these bounded deviations, thus greatly limiting the potential state space. The vertices are referred to as $h_{j,v}$, with $j \in [q] \cup \{0\}$ referring to the layer, specifically how many columns we have chosen so far, and v being a partial-sum vector. The graph is constructed so that paths correspond to sequences of column choices. In particular, a shortest path between the designated start and end vertices corresponds to an optimal solution of the n -fold ILP, and vice versa [3], which is easily achieved using *BFS*. For our purpose, we can simply check if such a path exists at all to determine feasibility.

3.1.4.2 Implementation

The implementation of this algorithm was made a bit easier due to the fact that Gupta already provided pseudocode which explains the solving process for the n -fold ILP. Once more, we extract values that we need for processing. After obtaining $b^\uparrow, \Delta$ and calculating q trivially by summing up the values $b_1^\downarrow, \dots, b_n^\downarrow$, we can perform what Gupta refers to as **Algorithm 1** to calculate M_σ . For this, we initialize our empty M_σ result array. We also declare the best imbalance found for the current iteration **bestImb** and the e value that corresponds to it **bestE**. Then, we work through each $j \in [q]$ to find the best imbalance value at each step. For this we set **bestE** to -1 and **bestImb** to $+\infty$. We calculate $\text{imb}_{M_\sigma}(e, j)$ by first determining $\text{occ}_{M_\sigma}(e, j)$ at each iteration step. The occurrences of e are found in linear time by simply checking the current state of M_σ in linear time $\mathcal{O}(n)$. The imbalance is then trivially determined in constant time with the formula that defines it. We then check if the current imbalance is smaller than **bestImb**. If yes, we update the best values accordingly. After we checked every $e \in [n]$ we set the resulting $M_\sigma[j]$ to **bestE**.

With M_σ determined, we can begin working on constructing the graph G . We represent the constructed layered DAG using a custom graph class. Each vertex is represented by its layer j and the corresponding vector v , while each edge represents the selection of a variable. Vertices are stored in adjacency lists and grouped by layer, allowing the DAG to be processed in topological order. To avoid duplicate representations of the same logical state, i.e. adding the same vector twice, vertices are canonicalized using a hash map.

We then followed **Construction 1** from the original paper by Gupta. We do this by first adding all relevant vertices to the graph. Specifically we add all vector $v \in \mathbb{Z}^r$, with $j \in [q]$ and $k \in [r]$ such that

$$(j/q)b_k^\uparrow - n\Delta(n+2r) \leq v_k \leq (j/q)b_k^\uparrow + n\Delta(1+2r).$$

To achieve this, we first calculate the lower and upper bounds on the values that each v_k can assume for every $j \in [q]$. We do this by using the above bounds, making sure to round the lower bounds up and the upper bounds down, as we do not work with fractional numbers. Then we generate every possible vector state in the given bounds. For this, we devised a function that, given the bounds, increments the last entry of the vector by 1. If this exceeds its upper bound, the next entry is incremented instead, while the previous entry is reset to its lowest possible value. This process is applied iteratively to the entries, thereby generating all possible states within the given bounds. We calculate the amount of vectors needed for a given $j \in [q]$ by summing up the differences between the upper and the lower bound for each position $k \in [r]$ and adding 1. After all of the vectors are added, we finally add the zero vector $\mathbf{0}$ to layer 0, finishing the first part of **Construction1**.

We then move to add the edges of the graph. The paper by Gupta [3] states that for each $j \in [q]$, we look at $M_\sigma[j] = i$, and add the directed edge $(h_{j-1,v}, h_{j,v_2})$ to set of edges if $v_2 - v_1$ is the column $A_i[\cdot, k]$ for some $k \in [t]$. Naively implemented, this would require checking many invalid combinations that do not refer to any column of A_i . We can reverse the formula however, and calculate v_2 instead, by combining the columns of A_i with v_1 , drastically reducing the amount of possible vectors we would have to verify. We check all possible combinations of v_1 and $A_i[\cdot, k]$ with $k \in [t]$ at layer $j \in [q]$ and an edge between v_1 and their sum v_2 if the sum is still in bounds. Additionally we add the objective function weight $c[i * t + k]$ to the edge, so we can calculate the minimum cost in the end. After we went through every layer and added every edge, we can now perform *BFS*, a well known graph traversal algorithm. This function returns us the minimum-cost path if one exists, and None if no such path exists. We can directly extract the cost from the path now if we so desired, however, for checking feasibility, it was enough to check if the path existed at all.

3.1.5 Lars2

3.1.5.1 Description

Rohwedder's [4] algorithm tackles a scheduling problem about makespan minimization for uniform machines. The problem consists of assigning jobs with given processing times to machines with different speeds such that the maximum completion time over all machines is minimized. We have $n \in \mathbb{N}$ jobs with processing times $p_1, \dots, p_n$ and $m \leq n \in \mathbb{N}$ machines with speeds $s_1, \dots, s_m$. A job j requires p_j/s_i units of processing time if it is executed on machine i . Additionally, only one job is able to run on a machine at any given time. Informally, the goal is to find a distribution of jobs that minimizes the maximum time any machine has to run.

The particular structure of the problem allows it to be viewed as an n -fold ILP. Existing algorithms for n -fold ILPs can be applied to the scheduling problem, but their running time has a dependence of $p_{\max}^{O(r^2)}$, where $p_{\max}$ is the largest processing time. Rohwedder's algorithm to solve the n -fold ILP improves on this by solving it in $p_{\max}^{O(r)} \cdot n$.

The algorithm employs the following idea: let $b_S^\downarrow$ be the sum for all components in

$b^\downarrow$, formally $b_S^\downarrow = \sum_{i=1}^n b_i^\downarrow$. Let P be a partition of $\{1, \dots, n\}$. Each $S \in P$ corresponds directly to the columns of the blocks of our input matrix $\mathcal{A}$, i.e. for a matrix

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 1 & 2 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

we would have $P = \{\{1, 2\}, \{3, 4\}\}$ with the possible values for S being $\{1, 2\}$ and $\{3, 4\}$. The algorithm starts with $x = 0$ and goes through $b_S^\downarrow$ iterations, where in each iteration a single variable of x is incremented by one. Each x_i that gets increased corresponds to set S . To achieve this, we construct all pairs $(j/b_i^\downarrow, S)$ for all $i \in [n]$ and $j = 1, \dots, b_i^\downarrow$ and order them non-decreasingly by their first component. The resulting sequence after correct ordering is denoted by $(d_1, S_1), \dots, (d_{b_S^\downarrow}, S_{b_S^\downarrow})$. Through this method of sorting, it is ensured that progress towards b is made in a balanced way, which helps bound the number of right-hand sides to keep track of later. [4]. Additionally, the progress can be thought of as a time interval $[0, 1]$. The first value of each set represents the time in which we process the pair, with all of the sets being processed by time 1 [4].

Then, once all pairs are generated, we solve the ILP as a path problem in a layered graph, similar to the algorithm from Gupta [3]. The graph G consists of $b_S^\downarrow + 1$ sets of vertices $V_1, \dots, V_{b_S^\downarrow}$. A vertex V_k contains all possible right hand sides $b' \in \mathbb{Z}^r$ with $\|b' - d_k \cdot b\|_\infty \leq 4rn\Delta$. To build the edges, we look at combinations of vectors $v' \in V_k$ and $v'' \in V_{k+1}$, with b' and b'' being the corresponding right-hand sides respectively. If $b'' - b'$ corresponds to one of the columns of A , we add an edge. To determine feasibility, we find the longest path in G . If such a path exists, $Ax = b$ is feasible.

3.1.5.2 Implementation

The implementations once more begins by extracting the relevant variables that are required for the calculation. In particular, we also extract A from $\mathcal{A}$ and transpose it to a variable we named **AT** and calculate $b_S^\downarrow$. The transposing was done so we can more easily access A by the column index. We already know the size of each block which is part of our instance information, however, for this algorithm we also want to know, at which index each block starts. For we create a new array **blockStartIndices** and set a helper variable **startIdx** to 0. Then we loop through the size of the blocks, set the value of **blockStartIndices**[i] in the current iterations i to **startIdx** and increment **startIdx** with the current value of **blockStartIndices**[i]. This gives us the array where we can index the blocks by their starting position in A .

Afterwards, we begin constructing all pairs $(j/b_i^\downarrow, S)$. We do this with a helper class **Breakpoint** which acts as a tuple. The breakpoint stores the value of the calculation $j/b_i^\downarrow$ in its first component and the corresponding set S , which is equivalent to the block it came from, in the second component. As described in the original paper, we calculate every possible breakpoint $(j/b_i^\downarrow, S)$ for all $i \in [n]$ and $j = 1, \dots, b_i^\downarrow$, then we order them in a non-decreasing way. We simply opted to use the inbuilt

Java sorting method, which for objects utilized Timsort (citation needed). Timsort combines ideas of merge and insertion sort and has a worst-case time complexity of $\mathcal{O}(n \log(n))$.

Now we can begin working on the graph. We reuse the same **Graph** class that we already used in the algorithm by Gupta [3], as it provides us with all the functionality we require. In this case, let the k index in V_k refer to the layer of the graph G . As we want to find a path from the 0-vertex $\mathbf{0}$ of V_1 to the b -vertex of V_{t+1} and edges only connect vertices on different layers, we do not need to add anything but $\mathbf{0}$ to V_1 . After initializing G with this vertex, we go through $b_S^\downarrow$ iterations. In each iteration except for the last we extract the value for the next breakpoint d_{k+1} . In iteration $b_S^\downarrow$ we set the value to 1. For this graph, we can build vertices and edges simultaneously, as we do not need to add vertices that do not have any incoming edge in the first place. To construct the vertices and edges, we use a similar trick we used in a few times in the process of this paper already. Instead of checking all pairs $v' \in V_k$ and $v'' \in V_{k+1}$ and see if their corresponding right hand sides b', b'' corresponds to a column A_i from S_i with $A_i = b'' - b'$, we instead calculate $b'' = A_i + b'$ for all A_i in the current set/block, and see if it fulfills our bounding requirement of $||b'' - d_{k+1} \cdot b|| \leq 4rn\Delta$. We also add a second pruning condition, as we only work with non-negative input matrices. If there is a b_k'' for any $k \in [n]$ that is greater than b_k we can discard it immediately, as the right hand sides will only grow larger, thus never leading to a valid solution. Should both conditions be fulfilled, we add the corresponding vertex v'' to layer $k + 1$ and add a directed edge from v' to v'' .

As we are limiting ourselves to feasibility of the problem, the check for feasibility is quite simple. Recall that we started the construction for G from the $\mathbf{0}$ vertex as the only vertex in our first layer of the graph. That means every single vertex v in G is connected to $\mathbf{0}$, and thus, we can check for the existence of b in the final layer of the graph to determine if the graph is feasible, as a path to it must exist due to the way the graph is constructed. This solves the feasibility problem.

4

Conclusion

Bibliography

- [1] K. Jansen, K. Kahler, L. Piroton, and M. Tutas, “New Algorithm for Combinatorial n-Folds and Applications,” in *20th International Symposium on Parameterized and Exact Computation (IPEC 2025)*, A. Agrawal and E. J. van Leeuwen, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 358, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 15:1–15:15, ISBN: 978-3-95977-407-9. DOI: 10.4230/LIPIcs.IPEC.2025.15. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.IPEC.2025.15>.
- [2] K. Jansen and L. Rohwedder, “On integer programming and convolution,” *CoRR*, vol. abs/1803.04744, 2018. arXiv: 1803.04744. [Online]. Available: <http://arxiv.org/abs/1803.04744>.
- [3] S. Gupta, P. Jain, S. Seetharaman, and M. Zehavi, *A simple algorithm for combinatorial n-fold ilps using the steinitz lemma*, 2026. arXiv: 2507.03766 [cs.DS]. [Online]. Available: <https://arxiv.org/abs/2507.03766>.
- [4] L. Rohwedder, *Eth-tight fpt algorithm for makespan minimization on uniform machines*, 2025. arXiv: 2501.04859 [cs.DS]. [Online]. Available: <https://arxiv.org/abs/2501.04859>.